NEURAL NETWORKS

Análise de Dados em Informática

Licenciatura em Engenharia Informática

ISEP/IPP

Ana Maria Madureira
2024/2025

Fontes:
- Tom Mitchell, Machine Learning. McGraw-Hill, 1997.
- Catarina Silva e Bernardete Ribeiro, Aprendizagem Computacional em Engenharia, Imprensa da Universidade de Coimbra, 2018
- Chantal D. La and Daniel T. Larose, Data Science Using Python and R, John Wiley & Sons, Inc., 2019

DT, NN AND KNN

Support Vector Machines, aims to find the optimal hyperplane in an N-dimensional space to separate data points into different classes. The algorithm maximizes the margin between the closest points of different classes. While it can handle regression problems, SVM is particularly well-suited for classification tasks.

Decision Trees are among the most used and have been applied to a large number of fields (since medial diagnosis to credit risk management in banking

Neural networks are gaining popularity in the computer vision and pattern recognition field.

K-nearest neighbors models are still commonly and successfully being used is:

- in the intersection between computer vision, pattern classification, and biometrics(to make predictions based on extracted geometrical features).

- recommender systems (via collaborative filtering and outlier detection

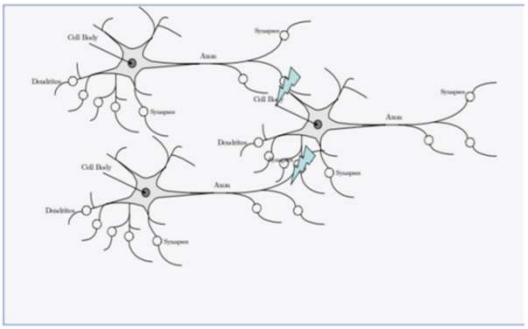
NEURAL NETWORK - HISTORIC PERSPECTIVE

1943 - McCulloch and Pitts proposed the model of an artificial neuron, also called the McCulloch-Pitts model
1948 - Weiner published the book "Cybernetics", in 1961, where aspects of learning and self-organization were discussed
1949 - Hebb published the book "The Organization of Behavior", in which Hebb's Rule was proposed
1958 - Rosenblatt introduced the simple one-layer network - Perceptron
1969 - Minsky and Paper published the book "Perceptrons" in which they demonstrated the limitations of a single layer network. The area then had a period of stagnation
1982 - Hopfield published several works dedicated to the Hopfield Recurring Networks
1982 - Kohonen developed the self-organizing maps with his name
1986 - The backpropagation algorithm for multi-layer networks was rediscovered, since then there has been a notable growth in the area
1985 - Boltzmann machines were proposed by Ackley, Hinton and Sejnowski
1989 - Yann LeCun proposed convolutional neural networks (CNN), initially advocated and developed by Fukushima in 1980
1992 - Neal moves forward with Bayesian networks
1995 - Recurring long memory networks were presented by Jurgen Schmidhuber
2006 - The appearance of Deep Neural Networks launched a new interest in the area with numerous applications
2014 - Adverse networks appeared by GoodFellow

70

70

NEURAL NETWORKS



Real neuron versus artificial neuron model

NN models are inspired by biological neuron structures and computations.

71

71

NEURAL NETWORKS

- Artificial neural networks have a surprising number of characteristics observed in the human cognitive process: learning by experience; generalization from examples and abstraction of the essential characteristics of information that contains irrelevant facts.
- Learning can be defined as the ability to perform new tasks that could not have been done before, or to improve previous performance as a result of changes produced by the learning process.
- Artificial neural networks can modify their behavior in response to stimuli produced by the environment: regulating the intensity of the connection between the processing units; adapting the synapse weights; recognizing the information presented to their neurons.
- To build a neuronal network it is necessary to consider the following mains steps:
  - Define the number and type of neurons;
  - Define how they connect (topology);
  - Start the connection weights;
  - Learn which weights are appropriate for the problem (learning)

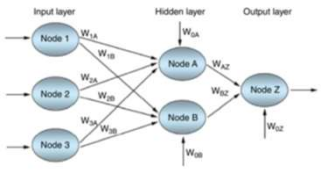
72

72

NEURAL NETWORKS

A neural network consists of a layered, feedforward, completely connected network of artificial neurons or nodes:

- The feedforward nature of the network restricts the network to a single direction of flow and does not allow looping or cycling.
- Most networks consist of three layers: an input layer, a hidden layer, and an output layer.
  - There may be more than one hidden layer, although most networks contain only one, which is sufficient for most purposes.
- The neural network is completely connected, meaning that every node in a given layer is connected to every node in adjoining layers, although not to other nodes in the same layer.
  - Each connection between nodes has a weight (e.g. W1A) associated with it.
  - At initialization, these weights are randomly assigned to values between 0 and 1.



- The number of hidden layers and the number of nodes in each hidden layer are both configurable by the analyst. How many nodes should one have in the completely connected network of artificial neurons or nodes.
- The feedforward nature of the network restricts the network to a single direction of flow and does not allow looping or cycling.

How many nodes should one have in the hidden layer?

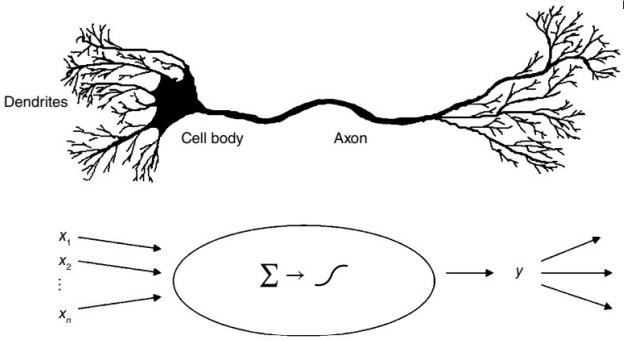
- Having more nodes in the hidden layer increases the power and flexibility of the network for identifying complex patterns – so, might be tempted to have a large number of nodes in the hidden layer.
  - an overly large hidden layer leads to overfitting, memorizing the training set at the expense of generalizability to the validation set
  - if overfitting is occurring, may consider to reduce the number of nodes in the hidden layer.
  - if the training accuracy is unacceptably low, may consider increasing the number of nodes in the hidden layer.

73

73

NEURAL NETWORKS

Neural networks represent an attempt at a very basic level to imitate the type of nonline learning that occurs in the networks of neurons found in nature, such as the human brain



Real neuron versus artificial neuron model

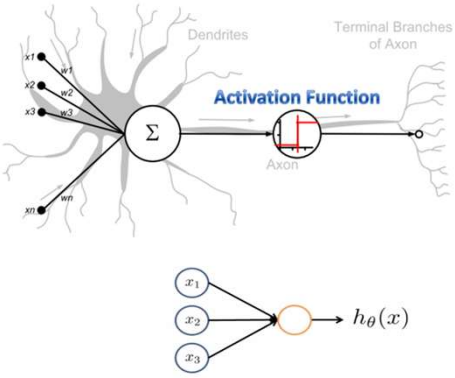
74

74

NEURAL NETWORKS – NEURON MODEL

An artificial neuron (similar to the biological) has the function of performing a simple operation that consists of receiving signals from the input connections and calculate a new output value that is and sent out. Two phases can be identified:

- Calculate the weighted sum (combination function) of the input values;
- Calculate the neuron activation value using a function - the activation function or transfer function, which has an input argument - the value calculated in the previous step.

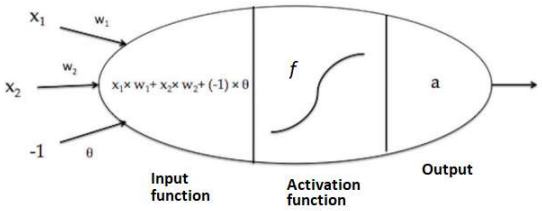


Model of artificial neuron (unit, code)

75

75

ARTIFICIAL NEURON - MATHEMATICAL MODEL



f : activation function (or transfer function)
∑: input function or combination function
w : input weight
θ: bias or offset
x_1, x_2: inputs

- 2 inputs x_1, x_2 are visible with 2 associated weights w_1, w_2 and one input special value -1, whose weight is called offset or bias and whose weight it is usually represented by the letter b or the Greek letter θ.
- The combination function is the weighted sum of these inputs is carried out in the first step of the operation of the neuron resulting in $\sum = x_1 \times w_1 + x_2 \times w_2 + (-1) \times \theta$.
- the activation function *f*, has as objective to determine whether the previously calculated weighted sum is sufficient to activate the neuron. the most common functions are the step function (sometimes also referred to as a signal function when the lower value is -1 instead of 0), the linear function and the sigmoid function.

76

76

COMBINATION FUNCTION

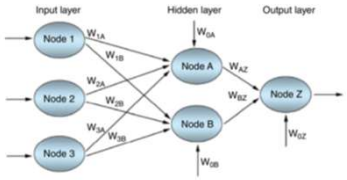
- The nodes in the hidden layer and the output layer collect the inputs from the previous layer and combine them using a combination function
- The combination function Σ produces a linear combination of the node inputs and the connection weights into a single scalar value, which we will term net. Thus, for a given node j, where x_{ij} represents the i^{th} input to node *j*, w_{ij} represents the weight associated with the i^{th} input to node *j*, and there are $I+1$ inputs to node j. Note that $x_1, x_2, \ldots, x_I$ represent inputs from upstream nodes, while x_0 represents a constant input, analogous to the constant factor in regression models, which by convention uniquely takes the value $x_{0j} = 1$. Thus, each hidden layer or output layer node *j* contains an "extra" input equal to a particular weight $W_{0j}\, x_{0j} = W_{0j}$, such as W_{0B} for node B

$$net_j = \sum_i W_{ij} x_{ij} = W_{0j} x_{0j} + W_{1j} x_{1j} + \cdots + W_{Ij} x_{Ij}$$

77

77

COMBINATION FUNCTION - EXAMPLE

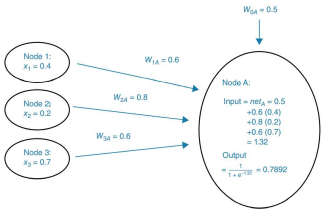


- Details of neural network, showing input to node A, combination function, and output from node A.
- we see that node A has combined its inputs into a net input of 1.32. For node A, this combination function netA = 1.32 is then used as an input to an activation function. In biological neurons, signals are sent between neurons when the combination of inputs to a particular neuron crosses a certain threshold and the neuron "fires." This is nonlinear behavior, since the firing response is not necessarily linearly related to the increment in input stimulation.

Data inputs and initial values for neural network weights

$x_0 = 1.0$	$W_{0A} = 0.5$	$W_{0B} = 0.7$	$W_{0Z} = 0.5$
$x_1 = 0.4$	$W_{1A} = 0.6$	$W_{1B} = 0.9$	$W_{AZ} = 0.9$
$x_2 = 0.2$	$W_{2A} = 0.8$	$W_{2B} = 0.8$	$W_{BZ} = 0.9$
$x_3 = 0.7$	$W_{3A} = 0.6$	$W_{3B} = 0.4$	—

$$net_A = \sum W_{iA} x_{iA} = W_{0A}(1) + W_{1A} x_{1A} + W_{2A} x_{2A} + W_{3A} x_{3A}$$
$$= 0.5 + 0.6(0.4) + 0.80(0.2) + 0.6(0.7) = 1.32$$



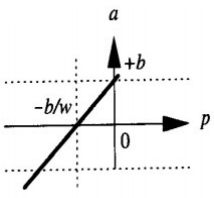
Details of neural network, showing input to node A, combination function, and output from node A.

78

78

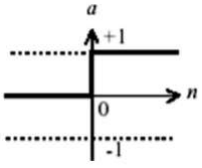
ACTIVATION FUNCTIONS

Linear



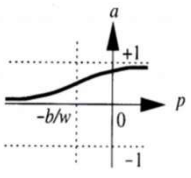
The bias b makes horizontal displacements of the linear function

Step



The bias b makes horizontal displacements of the step function

Sigmoid



$a = logsig(wp + b)$

The bias b makes horizontal displacements of the sigmoid function

79

Feed-forward network

In general, a neuron, even with several inputs, is not enough to solve the majority of real problems. Several neurons are required to work in parallel, in what is usually called a layer of neurons, or even several layers of neurons constituting a multilayered architecture

A network with only forward or direct connections, with three layers of neurons, with the input layer ($[x_1\ x_2]$). Each layer has its weight matrix w, the bias is not represented at this case.

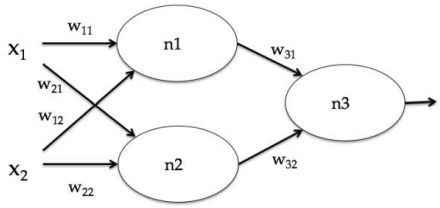
The output layer has only one neuron (n_3) and the intermediate layer, called a hidden layer, it consists of two neurons (n_1 and n_2), the various layers can have a different number of neurons.

When input layer and the output layer are all called the hidden layer (1st hidden layer, 2nd hidden layer, and so on from the input to the output). The outputs of the input layer constitute the inputs of the hidden layer, and so on, so that each layer can be interpreted as an isolated layer.

Faced with a problem to be solved with feed-forward neural networks, the number of neurons in the input and output layers are usually easy to define by the problem definition itself.

The number of neurons in each hidden layer and the number of hidden layers is more subjective. There are many heuristics to choose the structure of a neural network, there is no method that allows to choose exactly the best structure for a neural network.

If the network is too small, you may not be able to solve the proposed problem. While if the network is too large, despite being able to memorize the examples presented, it may prove unable to generalize beyond those examples - it may lose its most important capacity - the ability to learn (overfitting)

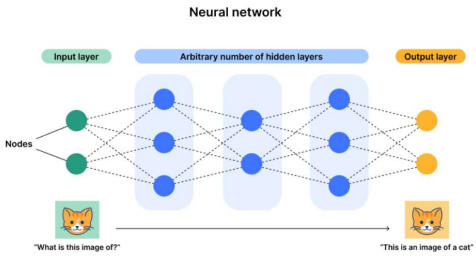


Model of a neuronal network with three layers of neurons

NEURAL NETWORK

The structure of an artificial neural network consists of:

- **Input layer**: takes in the input data and transfers it to the second (hidden) layer of neurons using synapses. An input layer has as many nodes as features or columns of data in the matrix.
- **Hidden layer**: takes data from the input layer to categorize or detect desired aspects of the data. Nodes in the hidden layer send the data to more hidden layers or, finally, to the output layer. The hidden layer of an ANN is a "black box" because researchers cannot determine its results.
- **Output layer**: takes data from the hidden layer and outputs the results. It has as many nodes as the model desires.
- **Synapses:** connect nodes in layers and in between layers.



Source: https://www.cloudflare.com/learning/ai/what-is-neural-network/

NEURAL NETWORK TOPOLOGIES

- There are several topologies of artificial neural networks with specific characteristics:
  - networks have only forward connections (feed-forward networks)
  - if there are feedback loops (recurrent networks).
  - have a supervisor or teacher, that is, if they are supervised or if they have autonomous learning algorithms(self-organized).

82

82

MULTILAYER NNS

- Multilayer NNs are more powerful than monolayer networks, having more applications. For example, unlike a monolayer network, a network with three layers, with sigmoid activation function in the hidden layer and linear activation function in the output layer, it is able to approximate any function with a small error, provided that properly trained.

83

83

THE BINARY PERCEPTRON: TRAINING AND LEARNING

- Learning rule or training algorithm: - a systematic procedure to modify the weights and the bias of a NN such that it will work as we need
- The most common learning approaches:
  - supervised learning - A set of Q examples of correct behavior of the NN is given (inputs p, outputs t)
  - reinforcement learning - receives only a classification that stimulate good performances.
  - unsupervised learning - has only inputs, not outputs, and learns to categorize them (dividing the inputs in classes, as in clustering)

84

84

THE BINARY PERCEPTRON:

- The perceptron learning rule is a supervised one
- It is simple, but powerful
- With one single layer, the binary perceptron can solve only linearly separable problems.
- The problems non linearly separable can be solved by multilayer architectures.

85

85

BACKPROPAGATION

- **How does the neural network learn?**
- As each observation from the training set is processed through the network, an output value is produced from the output node.
- This output value is then compared to the actual value of the target variable for this training set observation and the error (actual – output) is calculated. This prediction error is analogous to the prediction error in linear regression models. To measure how well the output predictions fit the actual target values, most neural network models use the sum of squared errors (SSE):

$$\text{SSE} = \sum_{\text{records}} \sum_{\text{output nodes}} (\text{actual} - \text{output})^2$$

- where the squared prediction errors are summed over all the output nodes and over all the records in the training set.
- The problem is to construct a set of model weights that will minimize the SSE. In this way, the weights are analogous to the parameters of a regression model. The "true" values for the weights that will minimize SSE are unknown and our task is to estimate them, given the data.

86

86

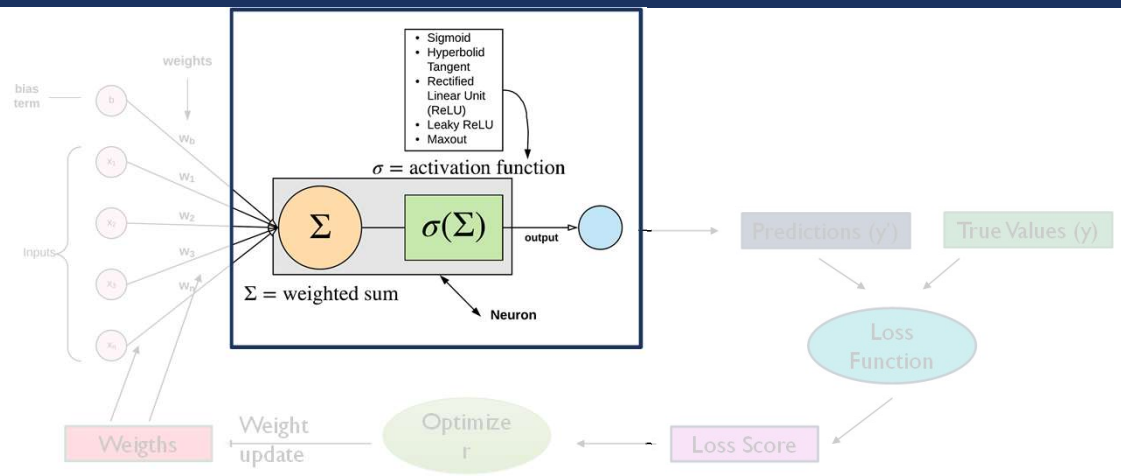
BACKPROPAGATION – ALGORIHTM

- The backpropagation algorithm does the following:
  - It takes the prediction error (actual – output) for a particular record and percolates the error back through the network.
  - Assign partitioned responsibility for the error to the various connections.
  - The weights on these connections are then adjusted to decrease the error, using gradient descent.

87

87

NEURAL NETWORK



88

ACTIVATION FUNCTIONS

- Introduction to Activation Functions
- Purpose and Importance in Deep Learning Models
- Characteristics
- Commonly Used Activation Functions, Strengths & Weaknesses:
  - Linear
  - Step
  - Sigmoid
  - Tanh
  - ReLU (Rectified Linear Unit)
  - Softmax (for multi-class classification)

89

INTRODUCTION TO ACTIVATION FUNCTIONS

- Activation functions are mathematical functions applied to the output of each neuron in a neural network.
- They introduce non-linearity to the network, allowing it to learn complex patterns and relationships in data.
- These functions determine whether a neuron should be activated or not, based on the weighted sum of its inputs.

90

PURPOSE AND IMPORTANCE IN DEEP LEARNING MODELS

- An activation function is a mathematical function applied to the output of a neuron. It introduces non-linearity into the model, allowing the network to learn and represent complex patterns in the data
- Activation functions are crucial for the successful training of neural networks.
- They enable networks to learn complex functions and make them capable of approximating any arbitrary function.
- Without activation functions, neural networks would reduce to linear regression models, limiting their expressiveness and learning capacity.

91

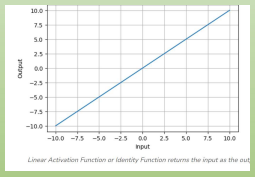
CHARACTERISTICS

- **Non-linearity:** Activation functions introduce non-linearities, enabling neural networks to approximate non-linear functions.
- **Differentiability:** Activation functions should be differentiable to facilitate the optimization process during training using gradient-based methods.
- **Range:** Activation functions typically have a specific output range, which can affect the behavior of the network and the stability of training.

92

LINEAR ACTIVATION FUNCTION

Defined as:
$f(x) = x$



Linear Activation Function or Identity Function returns the input as the out

Applicability
- Regression tasks where the output is expected to be a linear combination of input features.
- Simple linear models or shallow networks where non-linearity is not a requirement.
- Output layer in neural networks for regression tasks, predicting continuous values.

Strengths
- Direct mapping of input to output, preserving linearity.
- Suitable for tasks where a linear relationship between input and output is desired.
- Used in the output layer for regression tasks, providing continuous output values.

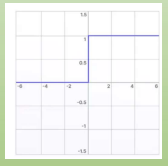
Weaknesses
- Lacks non-linearity, limiting the expressiveness of the network.
- Not effective for capturing complex patterns or non-linear relationships in the data.
- Prone to gradient vanishing or exploding problems in deep networks.

93

STEP ACTIVATION FUNCTION

Defined as:

$$f(x) = \begin{cases} 1, & x < 0 \\ 0, & x \geq 0 \end{cases}$$



Applicability

- introduce non-linearity, allowing networks to classify data into discrete categories.
- compares the input to a threshold; activation occurs if the input exceeds it; otherwise, the neuron gets deactivated, preventing its output from being passed on to the following hidden layer.
- the binary step function can be useful for simple neural networks where a basic decision-making process is needed, such as early binary classification tasks.
- is suitable for binary classifications.

Strengths

- Simple to implement.
- Useful for binary classification tasks.

Weaknesses

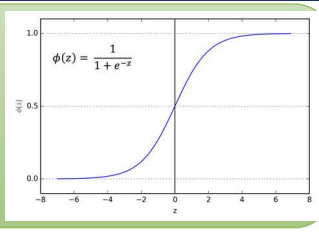
- Non-differentiable: Since the binary step is abrupt, it lacks a slope, which makes it unsuitable for gradient-based learning methods like backpropagation.
- Limited output: It can only handle binary outputs, making it inadequate for tasks requiring multi-class classification or regression.

94

SIGMOID ACTIVATION FUNCTION

Defined as:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$



Applicability

- Binary classification problems where outputs need to be interpreted as probabilities.
- Output layer for binary classification tasks.
- Logistic regression.

Strengths

- Smooth and bounded output between 0 and 1, mimicking probabilities.
- Useful in binary classification tasks where the output needs to be interpreted as probabilities. For **binary classification, it is also more efficient (compared to softmax)**.
- Well-suited for outputs that need to be in the range (0, 1).

Weaknesses

- Sigmoid saturates and kills gradients during backpropagation, known as the vanishing gradient problem.
- Outputs are not zero-centered, making optimization slower, especially in deep networks.
- Prone to the "vanishing gradient" problem, leading to slow convergence in deep networks.

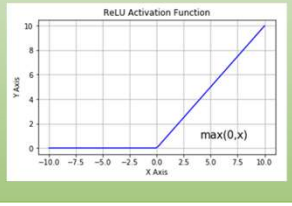
95

RELU ACTIVATION FUNCTION

Defined as:

$f(x) = \max(0, x)$

"0" is a threshold that can be configured, but defaults to 0



Applicability

- Hidden layers in deep neural networks, especially in convolutional neural networks (CNNs).
- Rectification of feature maps in CNNs.

Strengths

- Simple and computationally efficient.
- Overcomes the vanishing gradient problem by avoiding saturation in positive regions.
- Fast convergence in deep networks due to linear, non-saturating activation.

Weaknesses

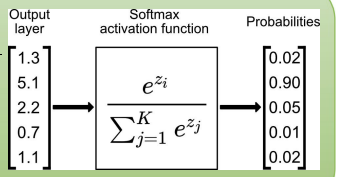
- Not zero-centered, which can lead to dead neurons (neurons that never activate).
- Prone to the "dying ReLU" problem where neurons can get stuck in the negative side and never recover.

96

SOFTMAX ACTIVATION FUNCTION

Defined as:

$$softmax(x_i) = \frac{e^{x_i}}{\sum_{j=1}^{N} e^{x_j}}$$



Applicability

- Multi-class classification problems where the output needs to represent class probabilities.
- Output layer in neural networks for multi-class classification tasks.

Strengths

- Converts raw scores into probabilities that sum up to 1, making it suitable for multi-class classification.
- Outputs can be interpreted as class probabilities, aiding in decision-making.
- Used in the output layer of neural networks for multi-class classification tasks.

Weaknesses

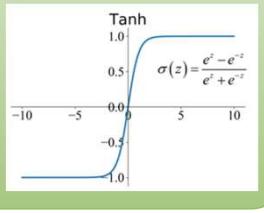
- Sensitive to large input values, which can lead to numerical instability.
- Requires careful consideration of loss functions, especially for imbalanced datasets.

97

TANH ACTIVATION FUNCTION

Defined as:

$tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$



Applicability

- Hidden layers in feedforward neural networks.
- Recurrent neural networks (RNNs) where zero-centered outputs are preferred.

Strengths

- Zero-centered outputs (-1 to 1), making optimization easier compared to sigmoid.
- Saturates less quickly than the sigmoid function, alleviating the vanishing gradient problem to some extent.
- Well-suited for outputs that need to be in the range (-1, 1).

Weaknesses

- Still prone to the vanishing gradient problem, especially in deep networks.
- Outputs are not always interpretable as probabilities.

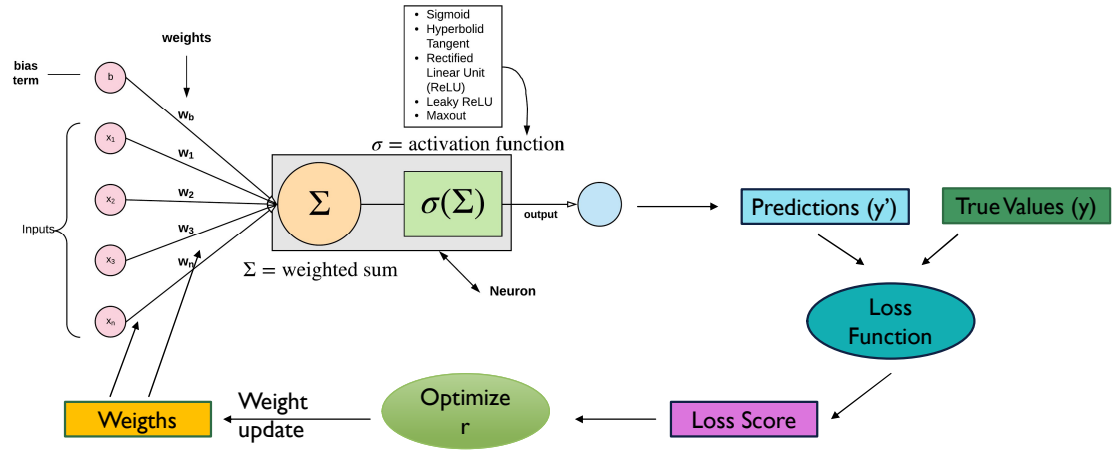
98

INDEX

- **Loss Functions**
- **Internal Parameters (weights & Bias)**

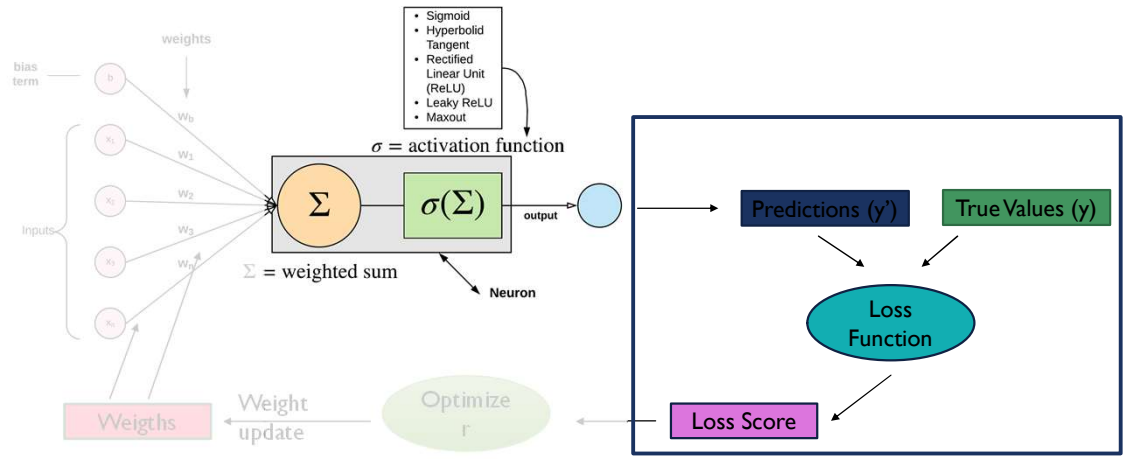
99

NEURAL NETWORK



100

LOSS FUNCTIONS



101

LOSS FUNCTIONS

- Introduction to Loss Functions
- Purpose and Importance in Deep Learning Models
- Commonly Used Loss Functions, Strengths & Weaknesses:
  - Regression
    1. MSE(Mean Squared Error)
    2. MAE(Mean Absolute Error)
    3. Huber loss
  - Classification
    1. Binary cross-entropy
    2. Categorical cross-entropy
    3. Sparse categorical cross-entropy

102

INTRODUCTION TO LOSS FUNCTIONS

Loss functions provide a **measure of how well the model is performing** on the given task. They **quantify the difference** between **predicted** outputs and **actual targets**.

They serve as the objective function during both model training and validation phases.

The goal of training is to minimize the loss, adjusting model parameters to improve performance.

Similarly, during validation, the loss is used to assess the model's generalization to unseen data.

The **hyperparameters** are adjusted to minimize the average loss.

103

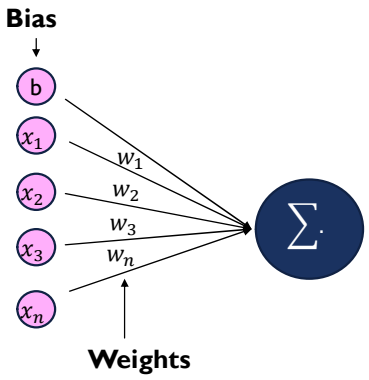
PURPOSE AND IMPORTANCE IN DEEP LEARNING MODELS

- Loss functions are crucial for the successful training of neural networks.
- They are used to evaluate model performance.

104

INTERNAL PARAMETERS - WEIGHTS

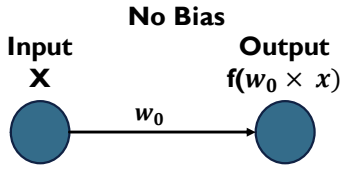
- Internal components or parameters of a Neural Network.
- They represent the strength of connections between neurons and convey the importance of each input feature in predicting the final output.
- During training, weights are optimized iteratively to minimize the difference between the predicted output and the true values.

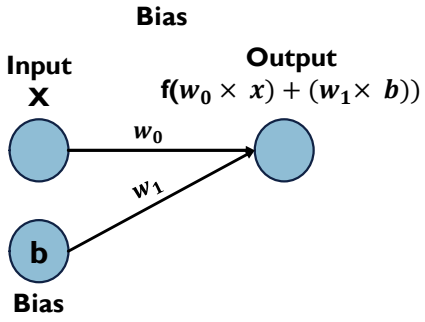


105

BIAS

- Shifts the activation function by adding a constant (i.e. the given bias) to the input.
- Ensures neurons can output non-zero values even when inputs are zero.





106

NEURAL NETWORKS - CONCLUSIONS

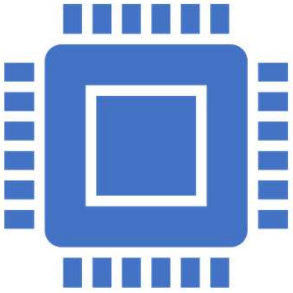
- The main benefit of neural networks is that they are quite robust for noisy, complicated, or nonlinear data, due to the nonlinear nature of the activation function.
- The main drawback of neural networks is that they are relatively opaque to human interpretation

107

107

ATIVIDADE: NEURAL NETWORKS

- As redes neuronais representam uma tentativa de imitar o quê?
- Qual é o principal benefício das redes neuronais para modelação?
- Descreva a principal desvantagem de uma rede neuronal.



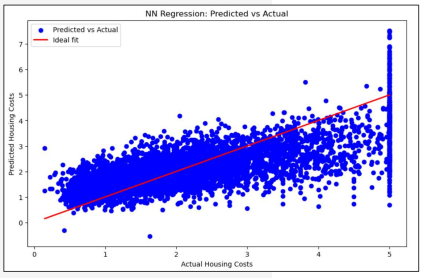

```python
from sklearn.neural_network import MLPRegressor
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score, mean_absolute_error, root_mean_squared_error, mean_squared_error
from sklearn.datasets import fetch_california_housing
import pandas as pd
import matplotlib.pyplot as plt
#
# Load California housing data set
#
housing = fetch_california_housing()
X = housing.data
y = housing.target
#=======================================================
print("Matriz de correlação")
#=======================================================
df = pd.DataFrame(data = housing.data,columns = housing.feature_names)
df["target"] = housing.target
matrix = df.corr()
print(matrix)
#df['species'] = housing.target_names[housing.target]# Summary statistics
#=================================================
# Create training/ test data split
#=================================================
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=1)
#=================================================
# Instantiate MLPRegressor
#=================================================
nn = MLPRegressor(
    activation='relu',
    hidden_layer_sizes=(10, 100),
    alpha=0.001,
    random_state=20,
    early_stopping=False
)
#=================================================
# Train the model
#=================================================
nn.fit(X_train, y_train)

import numpy as np
from sklearn.metrics import mean_squared_error

# Make prediction
pred = nn.predict(X_test)
print("Previsão:", pred)
```

```python
#=================================================
# Evaluate the model
#=================================================
mae=mean_absolute_error(y_test, pred)
mse = mean_squared_error(y_test, pred)
rmse = root_mean_squared_error(y_test, pred)
r2 = r2_score(y_test, pred)
print(f'Mean Absolute Error: {mae}')
print(f'Mean Squared Error: {mse}')
print(f'Root Mean Squared Error: {rmse}')
print(f'R-squared: {r2}')


# Visualize the results
plt.figure(figsize=(10, 6))
plt.scatter(y_test, pred, color='blue', label='Predicted vs Actual')
plt.plot([min(y_test), max(y_test)], [min(y_test), max(y_test)], color='red', linewidth=2, label='Ideal fit')
plt.title('NN Regression: Predicted vs Actual')
plt.xlabel('Actual Housing Costs')
plt.ylabel('Predicted Housing Costs')
plt.legend()
plt.show()
```




```
Matriz de correlação
              MedInc  HouseAge  AveRooms  AveBedrms  Population  AveOccup  \
MedInc      1.000000 -0.119034  0.326895  -0.062040    0.004834  0.018766
HouseAge   -0.119034  1.000000 -0.153277  -0.077747   -0.296244  0.013191
AveRooms    0.326895 -0.153277  1.000000   0.847621   -0.072213 -0.004852
AveBedrms  -0.062040 -0.077767  0.847621   1.000000   -0.066197 -0.006181
Population  0.004834 -0.296244 -0.072213  -0.066197    1.000000  0.069863
AveOccup    0.018766  0.013191 -0.004852  -0.006181    0.069863  1.000000
Latitude   -0.079809  0.011173  0.106389   0.069721   -0.108785  0.002366
Longitude  -0.015176 -0.108197 -0.027540   0.013344    0.099773  0.002476
target      0.688075  0.105623  0.151948   0.046701   -0.024650 -0.023737

              Latitude  Longitude    target
MedInc       -0.079809  -0.015176  0.688075
HouseAge      0.011173  -0.108197  0.105623
AveRooms      0.106389  -0.027540  0.151948
AveBedrms     0.069721   0.013344  0.046701
Population   -0.108785   0.099773 -0.024650
AveOccup      0.002366   0.002476 -0.023737
Latitude      1.000000  -0.924664 -0.144160
Longitude    -0.924664   1.000000 -0.045967
target       -0.144160  -0.045967  1.000000
Previsão: [1.94191688 1.30798781 2.28092569 ... 1.82630539 1.13557085 1.17275098]
Mean Absolute Error: 0.5967825334360669
Mean Squared Error: 0.6422687118732203
Root Mean Squared Error: 0.8014166905382121
R-squared: 0.5114649889444242
```

NEURAL NETWORKS – MLP CLASSIFICATION

```python
# Importing required libraries
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.datasets import load_iris
from sklearn.metrics import accuracy_score, classification_report, confusionMatrixDisplay, confusion_matrix
import matplotlib.pyplot as plt

# Loading dataset
irisData = load_iris()
X, y = irisData .data, irisData .target

# Splitting data set into train &amp;amp; test
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# Creating Object
scaler = StandardScaler()
# Standardizing the features
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Creating (MLP) classifier
clf = MLPClassifier(hidden_layer_sizes=(64, 32), max_iter=1000,
                    random_state=42)

# Training the model
clf.fit(X_train, y_train)
# Making prediction
y_pred = clf.predict(X_test)

# Determining Accuracy
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy*100:.2f}%")
```

classification_report() function provides a detailed report on the classification performance including precision, recall, F1-score and support for each class.

predict(): After training the model uses the learned weights and biases to make predictions on the test data (X_test). These predictions are stored in y_pred.

accuracy_score(): This function compares the predicted labels (y_pred) with the true labels (y_test) and computes the accuracy. Accuracy is the proportion of correctly predicted labels to the total number of labels.

```
Accuracy: 100.00%
```

```python
cm= confusion_matrix(y_test, y_pred)
print("_____Matriz de confusão 2 _____")
print(cm)
```

```
_____Matriz de confusão 2 _____
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
```

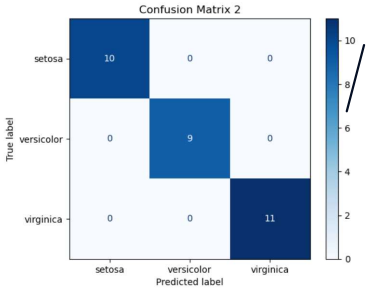
```python
print("_____Classification Report _____")
print(classification_report(y_test, y_pred))
```

```
_____Classification Report _____
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        10
           1       1.00      1.00      1.00         9
           2       1.00      1.00      1.00        11

    accuracy                           1.00        30
   macro avg       1.00      1.00      1.00        30
weighted avg       1.00      1.00      1.00        30
```

```python
from sklearn.metrics import accuracy_score, classification_report, ConfusionMatrixDisplay, confusion_matrix
cm= confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=irisData.target_names)
disp.plot(cmap=plt.cm.Blues)
plt.title('Confusion Matrix 2')
plt.show()
```



A confusion matrix is a table that summarizes the performance of a classification algorithm. It consists of four metrics:

True Positives (TP), True Negatives (TN), False Positives (FP), and False Negatives (FN).

110